



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2213 — Lógica para Ciencia de la Computación —
IIC2214 — Teoría de la Computación —

Pauta Interrogación 1

1er semestre 2025

Pregunta 1

- (a) (2.0 pts) Sea un lenguaje L sobre alfabeto Σ tal que su complemento $\bar{L}$ es finito. Demuestre que L es decidable.
- (b) (4.0 pts) Sea un lenguaje L sobre alfabeto Σ . Definimos el lenguaje *prefijo* de L como:

$$\text{prefix}(L) = \{w \in \Sigma^* \mid \text{existe } z \in \Sigma^* \text{ tal que } wz \in L\}.$$

Demuestre que si L es recursivamente enumerable, entonces $\text{prefix}(L)$ también lo es.

Solución

(a) Todo lenguaje finito $F = \{z_1, \dots, z_n\} \subseteq \Sigma^*$ es decidable. Una MT que decide F recuerda todas las palabras $z_1, \dots, z_n$, y dada una entrada $w \in \Sigma^*$, chequea que $w = z_i$ para algún i , en cuyo caso acepta; en caso contrario, rechaza. Como se vio en clases, si un lenguaje es decidable, entonces su complemento es decidable. Concluimos que L debe ser decidable.

Otra solución es dar directamente una MT que decide L : la MT recuerda todas las palabras $z_1, \dots, z_n$, y dada una entrada $w \in \Sigma^*$, chequea que $w \neq z_i$ para todo i , en cuyo caso acepta; en caso contrario, rechaza.

(b) Asumamos que L es recursivamente enumerable. Presentaremos tres posibles soluciones para demostrar que $\text{prefix}(L)$ es recursivamente enumerable.

Una primera solución puede ser usar no-determinismo. Como L es recursivamente enumerable, existe una MT M tal que $L = L(M)$. Podemos definir la siguiente MT no-determinista N :

Sobre entrada $w \in \Sigma^*$:

1. Adivinar una palabra $z \in \Sigma^*$ y escribir z después de w en la cinta.
2. Simular M sobre wz .
3. Si M acepta wz , **aceptar**.

Notar que el paso (1) efectivamente se puede implementar. Ponemos la cabeza lectora en el primer blanco después de w y entramos en un estado r . Desde r tenemos transiciones $(r, B, r, a, \triangleright)$, para cada $a \in \Sigma$ (las cuales nos permiten escoger algún símbolo en Σ y escribirlo en la cinta), y una transición $(r, B, s_0, B, \triangleleft)$, donde s_0 es el estado inicial del módulo que simula M sobre wz (la cual nos permiten escoger el final de la palabra z , y pasar a la simulación de M).

Veamos que $\text{prefix}(L) = L(N)$. Si $w \in \text{prefix}(L)$, entonces existe un $\hat{z} \in \Sigma^*$ tal que $w\hat{z} \in L$. En particular, M acepta $w\hat{z}$. Podemos tomar la ejecución de N sobre w , en la cual se escribe $\hat{z}$ después de w y luego se pasa a la simulación de M . Esta ejecución termina en aceptación, ya que M acepta $w\hat{z}$. Luego existe una ejecución de N sobre w que termina en aceptación, por lo cual $w \in L(N)$. Supongamos ahora que $w \in L(N)$. Tenemos que N acepta w , es decir, existe una ejecución que termina en aceptación. En esta ejecución se tuvo que haber escrito una palabra finita $\hat{z}$ después de w , y M tuvo que haber aceptado $w\hat{z}$. En particular, $w\hat{z} \in L$. Sigue que existe un $\hat{z}$ tal que $w\hat{z} \in L$, luego $w \in \text{prefix}(L)$.

Una segunda opción es usar el teorema de la Pregunta 3, visto en clases. Como L es recursivamente enumerable, existe un enumerador E tal que $L = L(E)$. Podemos definir el siguiente enumerador F que utiliza dos cintas de trabajo (cinta 1 y cinta 2) y una de impresión:

1. Simular E en la cinta 1 y cinta 2. La cinta 1 hace de cinta de trabajo de E y la cinta 2 hace de cinta de impresión de E .
2. Si E imprime una palabra x en la cinta 2, entonces **imprimir** todos los prefijos de x en la cinta de impresión.
3. Continuar la simulación de E .

Veamos que $L(F) = \text{prefix}(L)$. Si $w \in L(F)$, entonces F imprime w en la cinta de trabajo. Por definición de F , esto implica que w es un prefijo de un x que fue impreso por E , es decir, de un $x \in L$. Sigue que $w \in \text{prefix}(L)$. Por otra parte, asumamos que $w \in \text{prefix}(L)$. Luego existe z tal que $wz \in L$. Como $L(E) = L$, la palabra $x = wz$ es impresa en algún punto por E en el paso (2) de F . Como w es un prefijo de x , entonces F imprime w en el paso (3). Concluimos que $w \in L(F)$.

Una tercera opción es usar una MT determinista. Como L es recursivamente enumerable, existe una MT M tal que $L = L(M)$. Podemos construir una MT N que hace lo siguiente.

Sobre entrada $w \in \Sigma^*$:

1. Iterar sobre todos los naturales $i = 1, 2, 3, \dots$
2. Para cada i , simular i pasos de M sobre todas las palabras wz donde $z \in \Sigma^*$ y $|z| \leq i$.
3. Si M acepta algún wz en el paso (2), **aceptar**.
4. Pasar al siguiente natural i .

Veamos que $\text{prefix}(L) = L(N)$, lo cual implica que $\text{prefix}(L)$ es recursivamente enumerable. Si $w \in L(N)$, entonces N acepta w , y luego para algún i , en el paso (2) M acepta algún wz . En particular, existe un $z \in \Sigma^*$ tal que $wz \in L$, es decir, $w \in \text{prefix}(L)$. Por otra parte, supongamos que $w \in \text{prefix}(L)$, luego existe un $\hat{z} \in \Sigma^*$ tal que $w\hat{z} \in L$. En particular, M acepta $w\hat{z}$ en una cierta cantidad de pasos k . Veamos que la MT N acepta w a lo más en la iteración $i = \max\{k, |\hat{z}|\}$ (puede que N acepte w antes). En tal iteración i , se cumple que $i \geq |\hat{z}|$, luego la palabra $w\hat{z}$ es parte de los wz 's que se consideran en el paso (2), y además se cumple que $i \geq k$, luego la simulación de i pasos de M sobre $w\hat{z}$ debe aceptar. Concluimos que N acepta w , es decir, $w \in L(N)$.

Distribución de puntajes:

- (a) 2.0 pts describir una MT correcta que decida L .
- (b) 3.0 pts por describir una MT (o enumerador) que acepte (o enumere) correctamente $\text{prefix}(L)$, 1.0 pts por argumentar la correctitud de la construcción.

Se aplicarán descuentos a criterio del corrector.

Pregunta 2

Una MT con *reset a la izquierda* es igual que una MT usual, salvo que la función de transición tiene la forma:

$$\delta : (Q \setminus \{q_f\}) \times \Gamma \rightarrow Q \times \Gamma \times \{\nabla, \triangleright, \text{reset}\}$$

Si $\delta(q, a) = (q', b, \text{reset})$, entonces cuando la MT está en el estado q leyendo el símbolo a , actualiza su estado a q' , reemplaza a por b y la cabeza salta a la posición 0 de la cinta (posición donde está nuestro símbolo inicio $\vdash$). La dirección ∇ es quedarse en la misma posición.

Demuestre que para toda MT usual M , existe una MT con reset a la izquierda S tal que $L(S) = L(M)$.

(*Hint*: basta que piense cómo simular un movimiento a la izquierda de M , utilizando sólo los movimientos de S , que son quedarse, derecha y reset. Trate de utilizar marcas sobre los símbolos para detectar la posición a la izquierda de la cabeza.)

Solución

Sea una MT usual M . Debemos definir una MT con reset a la izquierda S tal que $L(S) = L(M)$. Siguiendo el hint, basta simular los movimientos a la izquierda de M . Es decir, por cada transición $\delta(q, a) = (q', b, \triangleright)$ en M , tenemos la misma transición en S , y por cada transición $\delta(q, a) = (q', b, \triangleleft)$ en M , tenemos en la MT S un módulo que hace lo siguiente: escribir en la posición actual el símbolo b , pasar a un módulo I que posiciona la cabeza lectora a la izquierda de la posición actual, y finalmente, actualizar el estado a q' . Basta describir el módulo I , lo cual hacemos a continuación.

Una primera idea es la siguiente: marcamos el símbolo c de la posición actual p con una marca especial, digamos que reemplazamos c por $\hat{c}$. Hacemos un reset a la izquierda, y luego nos movemos a la derecha hasta encontrar el símbolo marcado $\hat{c}$. El problema con esto, es que no sirve para detectar la posición $p - 1$, ya que una vez que vemos el símbolo marcado $\hat{c}$ en la posición p , ya nos hemos pasado de la posición $p - 1$. La idea entonces es iterativamente chequear acaso la posición $i = 0, 1, 2, 3, \dots$, es realmente la posición $p - 1$, es decir, la posición a la izquierda del símbolo $\hat{c}$. Para esto mantendremos dos símbolos marcados: el símbolo $\hat{c}$ de la posición p , y el símbolo $\hat{d}$ en la posición i que estamos testeando.

La simulación es la siguiente. En la posición p , marcar el símbolo actual c con $\hat{c}$. Luego hacer un reset a la izquierda, marcar el símbolo $\vdash$ en la posición 0 como $\hat{\vdash}$ y quedarse en la misma posición. (recordar que la MT S puede quedarse en la misma posición.) Ahora hacer iteraciones como sigue:

1. La cabeza está leyendo un símbolo marcado $\hat{x}$.
2. Chequear acaso $\hat{x}$ está a la izquierda de $\hat{c}$. Es decir, mover la cabeza a la derecha de $\hat{x}$ y verificar que el símbolo en esta posición también está marcado.
3. Si se cumple lo anterior, sabemos que encontramos la posición $p - 1$ que buscábamos (donde está el símbolo $\hat{x}$). Quitamos la marca en la posición $\hat{c}$ (reemplazar $\hat{c}$ por c), hacer reset a la izquierda, mover cabeza lectora a la derecha hasta encontrar el símbolo marcado $\hat{x}$, quitarle la marca a $\hat{x}$ (reemplazar $\hat{x}$ por x), y quedarse en la misma posición.
4. Si lo anterior no se cumple, mover la marca de $\hat{x}$ una posición a la derecha, y pasar a la siguiente iteración. Para hacer esto, hacer reset a la izquierda, moverse a la derecha hasta encontrar el primer símbolo marcado $\hat{x}$, reemplazar $\hat{x}$ por x , moverse una posición a la derecha, marcar el símbolo actual y con $\hat{y}$, quedarse en la misma posición, pasar al paso (1).

Notar que en nuestra simulación la MT S sólo se mueve a la derecha, se queda en la misma posición, o hace reset a la izquierda.

Distribución de puntajes:

6.0 pts por describir claramente una MT simuladora S correcta. Basta explicar cómo simular un movimiento a la izquierda de M . Se aplicarán descuentos a criterio del corrector.

Pregunta 3

Demuestre que un lenguaje L es recursivamente enumerable si y sólo si existe un enumerador E tal que $L = L(E)$.

Solución

($\Leftarrow$)

Sea un enumerador E tal que $L = L(E)$. La siguiente MT M cumple que $L(M) = L(E)$, luego, L es recursivamente enumerable.

Sobre entrada $w \in \Sigma^*$:

1. Simular E con 3 cintas, donde:
 - Cinta 1: siempre contiene la entrada w .
 - Cinta 2: simula la cinta de trabajo de E .
 - Cinta 3: simula la cinta de impresión de E .
2. La simulación de E se hace en las cintas 2 y 3.
3. Cada vez que E imprima una nueva palabra x en la cinta 3 comparamos la entrada w con x .
4. Si $w = x$, **aceptar**.
5. Si no, continuamos la simulación de E .

Veamos que $L(M) = L(E)$. Si $w \in L(M)$, entonces M acepta w en el paso (4). Sigue que en algún momento E imprimió una palabra x tal que $w = x$. Obtenemos que $w \in L(E)$. Supongamos ahora que $w \in L(E)$. Si ejecutamos M sobre w , tenemos que en algún momento E debe imprimir la palabra w en la cinta 3 y luego M acepta w en el paso (4). Concluimos que $w \in L(M)$.

($\Rightarrow$)

Sea una MT M tal que $L = L(M)$. El siguiente enumerador E cumple que $L(E) = L(M)$.

Enumerador E :

1. Iterar sobre todos los naturales $i = 1, 2, 3, \dots$
2. Para cada i , simular i pasos de M sobre todas las palabras w con $|w| \leq i$.
3. Cada vez que M acepte un w en el paso (2), **imprimir** w en la cinta de impresión.
4. Pasar al siguiente natural i .

Veamos que $L(E) = L(M)$. Si $w \in L(E)$, entonces en alguna iteración i , se cumple que $|w| \leq i$ y M acepta w en i pasos. En particular, M acepta w , y luego $w \in L(M)$. Asumamos ahora que $w \in L(M)$. Tenemos que M acepta w en cierta cantidad de pasos k . Veamos que E imprime w en la iteración $i = \max\{k, |w|\}$ (de hecho en cualquier iteración $i \geq \max\{k, |w|\}$). En tal iteración i , como $i \geq |w|$, en el paso (2) la palabra w será considerada en las simulaciones. Como $i \geq k$, la simulación de i pasos de M sobre w debe aceptar. Luego en el paso (3), E imprime w . Concluimos que $w \in L(E)$.

Distribución de puntajes:

- ($\Leftarrow$) 2.0 pts por presentar una MT M tal que $L(M) = L(E)$, 1.0 pts por argumentar que $L(M) = L(E)$.
- ($\Rightarrow$) 2.0 pts por presentar un enumerador E tal que $L(E) = L(M)$, 1.0 pts por argumentar que $L(E) = L(M)$.

Se aplicarán descuentos a criterio del corrector.